

聊聊Flink异步I/O机制的原理

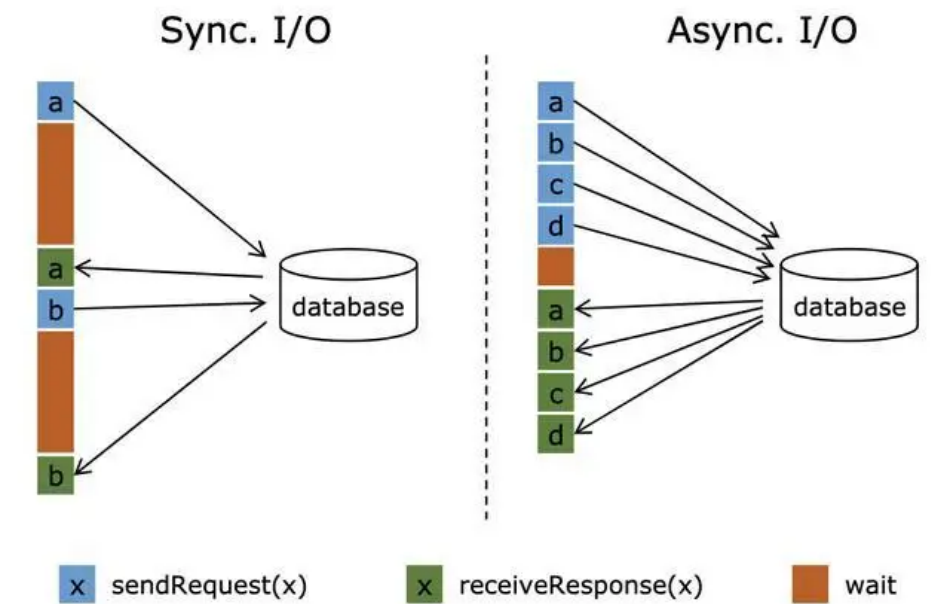
播报文章

高级互联网专家

2021-06-29 11:02 | 工具主管,内容评审官

关注

异步I/O专门用来解决Flink计算过程中与外部系统的交互问题。在默认情况下，算子向外部系统发出请求后即阻塞，等待结果返回才能发送下一个请求，可能会造成较大的延迟，吞吐量下降。有了异步I/O之后，就可以并发地发出请求和接收响应，延迟大大降低。下图来自官方文档，一看便知。



image

要享受异步I/O带来的便利，前提就是我们有一个能异步请求外部系统的客户端。如果原生没有提供，就得自己创建有限大小的线程池，将客户端放到线程池里调用。

异步I/O的原始设计文档见FLIP-12。但是随着时间的推移，它里面的内容与目前最新的Flink 1.9版本的实现有了一定出入，所以就不参考它了，直接讲讲笔者读过1.9版本的相关源码之后总结出的东西吧。

在调用AsyncDataStream.orderedWait()/unorderedWait()方法时，本质上是产生了一个AsyncWaitOperator算子，它是异步I/O的核心。每个AsyncWaitOperator都由三个主要的部分组成。

- AsyncFunction：执行异步操作的函数，用户需要覆写其asyncInvoke()方法并传入。
- StreamElementQueue：包含StreamElementQueueEntry的队列，底层由ArrayDeque实现。
- Emitter：单独的守护线程，将异步调用完成后的结果发送给下游算子。

作者最新文章

一份详尽的支付平台高可用架构设计实践

2小时前 6阅读

企业信息化BI项目的整体思想和架构

4小时前 38阅读

SpringCloud微服务概述及项目聚合实...

1天前 22阅读

相关推荐

Redis 的同步机制是什么？

Java技术汇

MySQL Filesort的优化

贵哥说Java创业

深入浅出讲解Java集合中的Collection...

IT千锋教育

C++编程基础：开发环境和工具...

尚硅谷教育

Github Copilot Chat的规则泄露...

deephub

百度热搜 >

换一换

1

笑果文化被暂停在京所有演出活动

2

Lisa刘亦菲赞达亚同框

3

古韵西安，共建美好未来

4

笑果被罚款1335万

5

笑果：即刻解除演员HOUSE合同

6

官方回应副镇长胁迫他人妻子

7

老人路边卖枇杷被城管撵 官方通报

8

孟羽童被开除？格力：祝一切都好

9

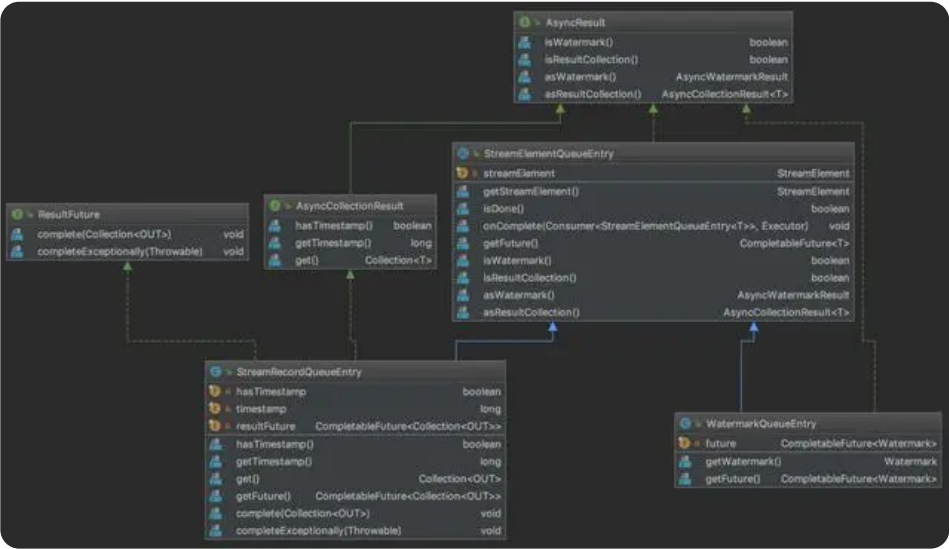
霍尊父亲称儿子结婚时间将近

10

在泰身亡中国商人妻子：他很少...

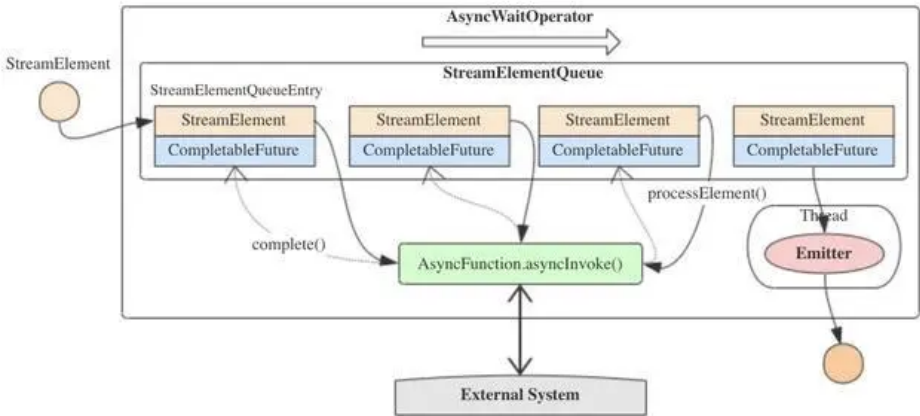
步返回。CompletableFuture是JDK 8提供的新特性，可以认为是非常好用的Future改进版，这里就不再展开讲了。

以下是以StreamElementQueueEntry为中心展开的类图。看官会注意到它有两种实现：代表数据的StreamRecordQueueEntry，和代表水印的WatermarkQueueEntry。它们都持有CompletableFuture。



image

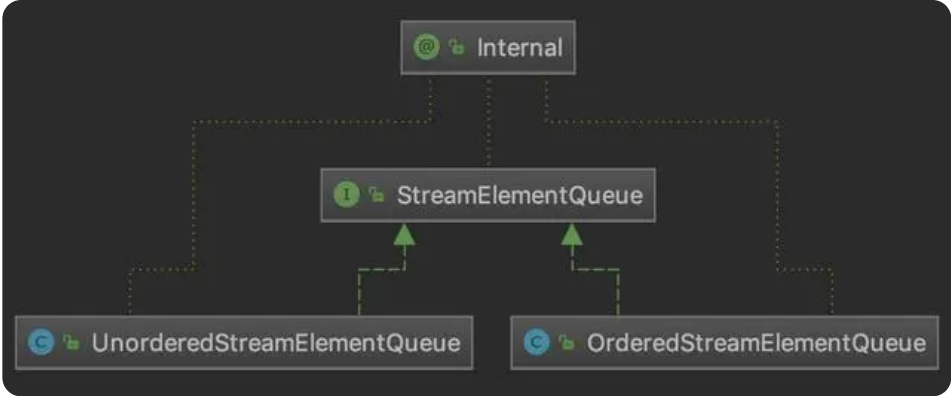
AsyncWaitOperator的机制可以用下面的简图来表示。



image

1. 来自上游的StreamElement进入AsyncWaitOperator的StreamElementQueue，并被封装成StreamElementQueueEntry。
2. AsyncWaitOperator调用传入的AsyncFunction的asyncInvoke()方法，该方法异步地与外部系统交互。
3. 异步操作完成后，由asyncInvoke()方法显式地调用ResultFuture.complete()方法，将结果返回；或者调用completeExceptionally()方法表示出现了异常。ResultFuture就是CompletableFuture的代理接口。
4. Emitter线程从StreamElementQueue中拉取那些已经完成了的StreamElementQueueEntry，并输出到下游算子。

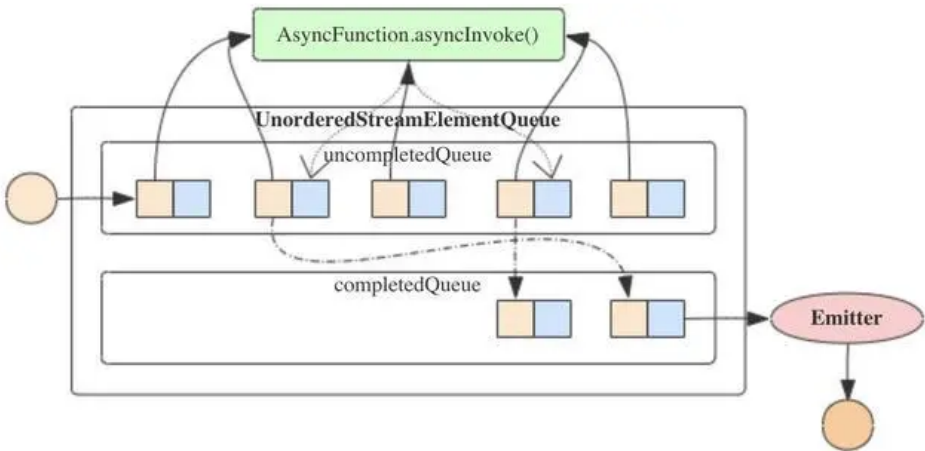
- 调用AsyncDataStream.orderedWait(): 创建OrderedStreamElementQueue队列，保持请求的顺序与输出结果的顺序相同，亦即先进先出。
- 调用AsyncDataStream.unorderedWait(): 创建UnorderedStreamElementQueue队列，不保持顺序。在采用处理时间时，先返回的结果先输出。而采用事件时间时，需要额外保证水印的边界不错乱。



image

简单讨论一下。

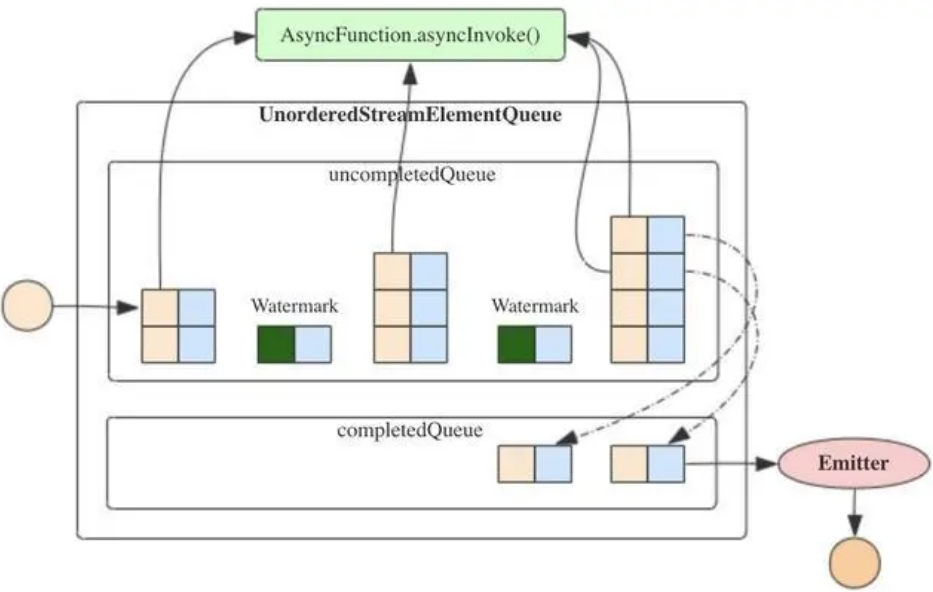
- **有序**
有序是最简单的情况，只需要将元素按照到来的顺序放入OrderedStreamElementQueue。只有当队列中的队头请求异步操作返回了结果，才会触发Emitter输出，后面的请求先返回也只能等待。
- **无序（处理时间）**
这种情况也不难办。在UnorderedStreamElementQueue中维护两个子队列，一个是未完成请求的队列（uncompletedQueue），一个是已完成请求的队列（completedQueue）。所有请求都先进入uncompletedQueue并执行异步操作，并按照操作完成的顺序进到completedQueue中。Emitter从completedQueue拉取并输出结果即可。如下图所示。



image

- **无序（事件时间）**
这是比较复杂的情况：我们允许两个水印之间的元素乱序，但是水印不能乱。所以在使用两个队列的同时，uncompletedQueue中还必须存储水印，这就是上面

从入来的异步请求的异步请求返回，并部分请求移动到completedQueue里面。这样就可以保证在通过某个水印之前，它前面的所有异步请求都完成。如下图所示。



异步I/O的检查点做起来很容易。由上面的分析可以知道，`StreamElementQueue`保存的就是尚未完成异步请求的元素，以及已完成异步请求但还没有送到Emitter发送的元素，只要遍历该队列，并将它们都放入状态后端就OK。

[举报/反馈](#)

大家都在搜

- flink异步io
- flink端到端一致性
- flink一个流控制另一个流
- flink工作原理
- flink原理、实战与性能优化
- flink join原理

发表评论

 发表神评妙论



发表